

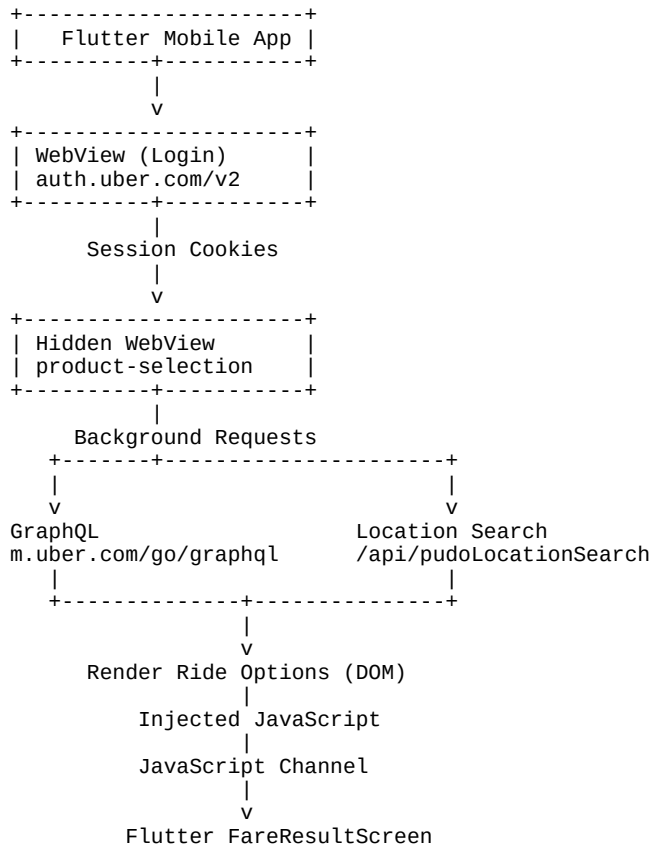
System Design Document (SDD)

Uber Fare Fetch Flow

1. Purpose

Describe the architecture for authenticating a user with Uber in a WebView, reusing the authenticated session, retrieving ride options through Uber's web experience, extracting fare information, and displaying it in Flutter.

2. High-Level Architecture



3. Components

Flutter App: UI, cookie persistence, navigation, JSON parsing.
WebView: Browser context for authentication and Uber pages.
Hidden WebView: Loads Product Selection without user interaction.
Uber Auth: Creates authenticated session.
Product Selection: Renders ride options.
GraphQL: Returns ride/product data for the authenticated session.
Location Search: Resolves pickup/drop into coordinates and place IDs.
Injected JavaScript: Extracts ride names and prices.
JavaScript Channel: Transfers extracted data to Flutter.

4. Sequence Flow

```
User -> Flutter : Select Uber
Flutter -> WebView : Open auth.uber.com/v2
User -> Uber : Login
WebView -> Flutter : Session Cookies
Flutter -> Hidden WebView : Restore Cookies
Flutter -> Hidden WebView : Load Product Selection URL
Hidden WebView -> Location API : Resolve locations (when required)
```

Hidden WebView -> GraphQL : Request products/fares
GraphQL -> Hidden WebView : Ride data
Injected JS -> Flutter : {displayName, price}
Flutter -> FareResultScreen : Display fares

5. Data Flow

Pickup/Drop -> Product Selection URL -> Authenticated WebView -> GraphQL Response -> DOM
-> Injected JavaScript -> JSON -> Flutter Models -> Fare Result UI.

6. Responsibilities

Flutter manages state and UI. WebView maintains authenticated browser session. Uber services authenticate, resolve locations, and compute fares. JavaScript extracts rendered information only.

7. Error Handling

Handle expired sessions by redirecting to login, network failures with retry, empty ride lists with user feedback, and JavaScript extraction failures with logging.